

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



HUỲNH NGUYỄN QUỐC BẢO - PHÍ NGỌC THÁI BÌNH

ỨNG DỤNG LLM ĐỂ PHÁT TRIỂN
ỨNG DỤNG TRẢ LỜI EMAIL TỰ ĐỘNG
DỰA TRÊN NỘI DUNG EMAIL ĐÃ GỬI

TÀI LIỆU HƯỚNG DẪN CÀI ĐẶT
BÁO CÁO THỰC TẬP TỐT NGHIỆP

THÀNH PHỐ HỒ CHÍ MINH, 2025

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



HUỖNH NGUYỄN QUỐC BẢO – 22H1120002

PHÍ NGỌC THÁI BÌNH – 22H1120095

ỨNG DỤNG LLM ĐỂ PHÁT TRIỂN
ỨNG DỤNG TRẢ LỜI EMAIL TỰ ĐỘNG
DỰA TRÊN NỘI DUNG EMAIL ĐÃ GỬI

TÀI LIỆU HƯỚNG DẪN CÀI ĐẶT
BÁO CÁO THỰC TẬP TỐT NGHIỆP

GIÁO VIÊN HƯỚNG DẪN:
TS. HÀN TRUNG ĐỊNH

NIÊN KHÓA 2022 - 2026

MỤC LỤC

1.	Giới thiệu chung.....	1
2.	Hướng dẫn cài đặt	1
2.1.	Môi trường chạy dự án	1
2.2.	Cài đặt backend & khởi chạy	1
2.2.1.	Yêu cầu hệ thống	1
2.2.2.	Cài đặt Python.....	2
2.2.3.	Clone Repository	3
2.2.4.	Cấu hình Supabase Database	3
2.2.5.	Cấu hình Gmail API.....	6
2.2.6.	Cài đặt Dependencies Python	8
2.2.7.	Cấu hình Environment Variables.....	10
2.2.8.	Khởi chạy Backend Server	12
2.3.	Cài đặt frontend & khởi chạy	13
2.3.1.	Yêu cầu hệ thống	13
2.3.2.	Các bước cài đặt.....	13
2.3.3.	Cấu hình biến môi trường	14
2.3.4.	Khởi chạy ứng dụng.....	15
2.3.5.	Kiểm tra cài đặt.....	15
2.3.6.	Lưu ý quan trọng.....	16
2.3.7.	Quy trình khởi chạy nhanh.....	16

1. Giới thiệu chung

Ứng dụng "Trợ lý Email Thông minh" được xây dựng với mục đích cung cấp một công cụ hỗ trợ tự động hóa quy trình trả lời email cho các nhân viên văn phòng và doanh nghiệp hiện đại. Trong bối cảnh khối lượng email trao đổi hàng ngày ngày càng gia tăng và trở thành gánh nặng về năng suất lao động, ứng dụng này giúp giải phóng thời gian quý báu của người dùng bằng cách tự động gợi ý các bản nháp trả lời thông minh, được cá nhân hóa dựa trên chính văn phong và lịch sử email của họ.

Hệ thống tích hợp công nghệ trí tuệ nhân tạo kết hợp với kỹ thuật RAG (Retrieval-Augmented Generation) để học hỏi từ những email mà người dùng đã gửi trong quá khứ, từ đó tạo ra các câu trả lời không chỉ phù hợp về mặt nội dung mà còn nhất quán về giọng điệu, văn phong, đảm bảo tính cá nhân hóa sâu và duy trì đặc trưng giao tiếp của từng cá nhân. Ứng dụng đóng vai trò là "trợ lý ảo" thông minh, luôn sẵn sàng hỗ trợ người dùng trong mọi tình huống giao tiếp qua email.

2. Hướng dẫn cài đặt

2.1. Môi trường chạy dự án

Dự án này có thể khởi chạy trên các IDE hoặc Text editor cài đặt kèm các extension cần thiết. Trong dự án này, để thuận tiện cho việc phát triển và quản lý mã nguồn, nhóm sử dụng Visual Studio Code. Visual Studio Code được khuyến nghị do hệ sinh thái extension phong phú. Song song đó, người dùng cũng cần cài đặt Git để có thể lấy được mã nguồn của dự án trọn vẹn.

2.2. Cài đặt backend & khởi chạy

2.2.1. Yêu cầu hệ thống

- Phần cứng tối thiểu:
 - + CPU: Intel Core i5 hoặc tương đương (khuyến nghị i7 hoặc cao hơn).
 - + RAM: 8GB/16GB.
 - + Ổ cứng: 20GB trống.
 - + GPU: Không bắt buộc nhưng nên có NVIDIA GPU với 8GB+ VRAM để tăng tốc độ xử lý AI.

- Phần mềm cần cài đặt:
 - + Hệ điều hành: Windows 10/11, macOS 11+, hoặc Linux (Ubuntu 20.04+).
 - + Python: Phiên bản 3.10 hoặc 3.11 (không dùng 3.12 vì có thể conflict với một số thư viện).
 - + Git: Để clone repository.

2.2.2. Cài đặt Python

- Đối với hệ điều hành Windows:
 - + Tải Python:
 - Truy cập: <https://www.python.org/downloads/> .
 - Tải Python 3.11.x (khuyến nghị).
 - Khi cài đặt, nhớ tick vào "Add Python to PATH".
 - + Kiểm tra cài đặt:

```
python --version
# Kết quả: Python 3.11.x

pip --version
# Kết quả: pip 23.x.x
```

- Đối với hệ điều hành macOS:

```
# Sử dụng Homebrew
brew install python@3.11

# Kiểm tra
python3 --version
pip3 --version
```

- Đối với hệ điều hành Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install python3.11 python3.11-venv python3-pip

# Kiểm tra
python3.11 --version
```

2.2.3. Clone Repository

Mở GitBash/Terminal/Powershell, chuyển đến nơi lưu thông qua lệnh `cd <Đường dẫn thư mục>`, sau đó lấy mã nguồn của dự án thông qua lệnh

```
git clone https://github.com/Harry1720/AppLLM\_AutoReplyEmail.git
```

Truy cập vào Visual Studio Code, mở dự án vừa clone lên, sau đó di chuyển vào nhánh backend thông qua lệnh

```
git checkout backend
```

Để xem dự án trên GitHub, truy cập đường dẫn:

https://github.com/Harry1720/AppLLM_AutoReplyEmail

2.2.4. Cấu hình Supabase Database

Supabase là backend-as-a-service cung cấp PostgreSQL database và Vector Store.

Bước 1: Tạo Project Supabase

- Truy cập: <https://supabase.com>.
- Đăng ký tài khoản (nếu chưa có) - có thể dùng GitHub để login nhanh.
- Click "New Project".
- Điền thông tin:
 - + Name: AppLLM-AutoReply (hoặc tên bạn thích).
 - + Database Password: Tạo mật khẩu mạnh (lưu lại để dùng sau).
 - + Region: Chọn gần Việt Nam nhất (Singapore hoặc Tokyo).
- Click "Create new project" - đợi ~2 phút để Supabase setup.

Bước 2: Lấy API Keys và URL

- Trong project dashboard, click biểu tượng Settings (bánh răng) ở sidebar trái.
- Chọn API trong menu.
- Copy và lưu lại:
 - + Project URL (ví dụ: <https://abc123xyz.supabase.co>).
 - + anon public key (dùng cho client-side).
 - + service_role key (quan trọng - dùng cho backend, có full quyền).

Bước 3: Tạo Database Tables

- Trong sidebar, click SQL Editor.
- Click "New query".
- Copy và paste script SQL sau, sau đó click Run:

```
-- Tạo bảng users
CREATE TABLE IF NOT EXISTS users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  picture TEXT,
  google_refresh_token TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Tạo bảng email_drafts
CREATE TABLE IF NOT EXISTS email_drafts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE
  CASCADE,
  email_id TEXT NOT NULL,
  thread_id TEXT NOT NULL,
  draft_id TEXT UNIQUE NOT NULL,
  subject TEXT NOT NULL,
  body TEXT NOT NULL,
  recipient TEXT NOT NULL,
  status TEXT DEFAULT 'draft',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

```

-- Tạo bảng documents (Vector Store)
CREATE TABLE IF NOT EXISTS documents (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id TEXT NOT NULL,
  content TEXT NOT NULL,
  metadata JSONB,
  embedding VECTOR(384),
  -- Dimension của sentence-transformers/all-MiniLM-L6-v2
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Tạo index cho tìm kiếm nhanh
CREATE INDEX IF NOT EXISTS idx_documents_user_id ON
documents(user_id);
CREATE INDEX IF NOT EXISTS idx_email_drafts_user_id ON
email_drafts(user_id);
CREATE INDEX IF NOT EXISTS idx_email_drafts_draft_id ON
email_drafts(draft_id);

-- Enable pgvector extension (cần cho vector search)
CREATE EXTENSION IF NOT EXISTS vector;

```

Bước 4: Tạo RPC Function cho Vector Search: Vẫn trong SQL Editor, tạo query mới và chạy:

```

-- Function để search documents theo vector similarity
CREATE OR REPLACE FUNCTION match_documents(
  query_embedding VECTOR(384),
  match_count INT DEFAULT 3,
  filter_user_id TEXT DEFAULT NULL
)
RETURNS TABLE (
  id UUID,

```



```
content TEXT,  
    metadata JSONB,  
    similarity FLOAT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
RETURN QUERY  
SELECT  
    documents.id,  
    documents.content,  
    documents.metadata,  
    1 - (documents.embedding <=> query_embedding) AS similarity  
FROM documents  
WHERE (filter_user_id IS NULL OR documents.user_id = filter_user_id)  
ORDER BY documents.embedding <=> query_embedding  
LIMIT match_count;  
END;  
$$;
```

Xác nhận: Sau khi chạy, vào Table Editor ở sidebar để xem các bảng users, email_drafts, documents đã được tạo.

2.2.5. Cấu hình Gmail API

Bước 1: Tạo Project trên Google Cloud Console

- Truy cập: <https://console.cloud.google.com/>.
- Đăng nhập bằng tài khoản Google.
- Click "Select a project" ở góc trên bên trái → "New Project".
- Đặt tên project: AppLLM-AutoReply (hoặc tên khác cũng được).
- Click "Create" và đợi vài giây.

Bước 2: Enable Gmail API

- Trong dashboard, click "Enable APIs and Services" (hoặc tìm trong menu).
- Tìm kiếm: "Gmail API".
- Click vào Gmail API trong kết quả.
- Click nút "Enable".
- Làm tương tự với "Google People API" (để lấy thông tin user).

Bước 3: Tạo OAuth 2.0 Credentials

- Trong sidebar trái, click "Credentials".
- Click "Create Credentials" → chọn "OAuth client ID".
- Nếu chưa cấu hình OAuth consent screen, bạn sẽ được yêu cầu làm trước:
 - + Click "Configure Consent Screen".
 - + Chọn "External" (cho phép bất kỳ ai test).
 - + Click "Create".
 - + Điền thông tin:
 - App name: AppLLM AutoReply (hoặc tên khác cũng được).
 - User support email: Email của bạn.
 - Developer contact: Email của bạn.
 - + Click "Save and Continue" → "Save and Continue" (bỏ qua Scopes) → "Save and Continue".
 - + Thêm email test của bạn vào Test users → "Save and Continue".
- Quay lại Credentials, click "Create Credentials" → "OAuth client ID".
- Chọn:
 - + Application type: Web application.
 - + Name: AppLLM Backend.
 - + Authorized redirect URIs: Thêm URL sau:
<http://localhost:3000/auth/callback>.
- Click "Create".

Bước 4: Download Credentials

- Sau khi tạo, một popup hiện ra với Client ID và Client Secret.
- Click "Download JSON".
- Đổi tên file thành credentials.json.
- Di chuyển file này vào thư mục gốc của backend (cùng cấp với app/).

Cấu trúc file credentials.json:

```
{
  "web": {
    "client_id": "xxx.apps.googleusercontent.com",
    "project_id": "your-project",
    "auth_uri": "https://accounts.google.com/o/oauth2/auth",
    "token_uri": "https://oauth2.googleapis.com/token",
    "client_secret": "YOUR_SECRET",
    "redirect_uris": ["http://localhost:3000/auth/callback"]
  }
}
```

2.2.6. Cài đặt Dependencies Python

Bước 1: Tạo Virtual Environment: Điều này giúp cách ly dependencies của dự án với hệ thống.

- Đối với hệ điều hành Windows:

```
# Di chuyển vào thư mục dự án (nếu chưa)
cd AppLLM_AutoReplyEmail
# Tạo virtual environment
python -m venv venv
# Kích hoạt virtual environment
.\venv\Scripts\activate
# Dấu hiệu thành công: (venv) xuất hiện trước command prompt
```

- Đối với hệ điều hành macOS/Linux:

```
# Tạo virtual environment
python3.11 -m venv venv
# Kích hoạt
source venv/bin/activate
# Dấu hiệu: (venv) xuất hiện trước $
```

Bước 2: Upgrade pip

```
pip install --upgrade pip
```

Bước 3: Cài đặt Dependencies

- Nếu có file requirements.txt trong dự án:

```
pip install -r requirements.txt
```

- Nếu không có, tạo file requirements.txt trong thư mục gốc với nội dung sau:

```
fastapi==0.109.0
uvicorn[standard]==0.27.0
python-dotenv==1.0.0
pydantic==2.5.0
pydantic-settings==2.1.0
# Google APIs
google-auth==2.27.0
google-auth-oauthlib==1.2.0
google-auth-http2==0.2.0
google-api-python-client==2.115.0
# Database
supabase==2.3.0
psycopg2-binary==2.9.9
# AI & ML
langchain==0.1.0
langchain-community==0.0.13
```

```
langchain-core==0.1.10
langgraph==0.0.20
langchain-huggingface==0.0.1
sentence-transformers==2.3.1
# Utilities
httpx==0.26.0
pyjwt==2.8.0
```

Sau đó chạy:

```
pip install -r requirements.txt
```

- Thời gian cài đặt: 5-10 phút tùy tốc độ mạng.

Bước 4: Kiểm tra cài đặt

```
python -c "import fastapi; import langchain; import supabase; print('All
packages
installed successfully!')"
```

Nếu không có lỗi, bạn đã sẵn sàng.

2.2.7. Cấu hình Environment Variables

Bước 1: Tạo file .env

- Trong thư mục gốc của backend (cùng cấp với app/), tạo file .env:
 - + Windows (PowerShell):


```
New-Item -Path .env -ItemType File
```
 - + macOS/Linux:


```
touch .env
```

Bước 2: Thêm các biến môi trường: Mở file .env bằng text editor (Notepad++, VSCode, v.v.) và thêm nội dung sau:

```
# Lấy từ file credentials.json bạn đã download
GOOGLE_CLIENT_ID=your-client-id.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=your-client-secret
```

```
# Lấy từ Supabase dashboard > Settings > API
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_KEY=your-service-role-key-here

# JWT SECRET (Tạo random string), dùng để mã hóa JWT token
# Tạo bằng lệnh: python -c "import secrets; print(secrets.token_urlsafe(32))"
JWT_SECRET=your-random-secret-key-at-least-32-characters-long

# Lấy mô hình llama-3.3-70b-versatile
GROQ_API_KEY=your-groq-api-key
GROQ_MODEL=llama-3.3-70b-versatile

# Môi trường: development hoặc production
ENV=development

# CORS: Cho phép frontend kết nối
CORS_ORIGINS=http://localhost:3000,http://localhost:3001
```

Bước 3: Điền thông tin thực

- Lấy GOOGLE_CLIENT_ID và GOOGLE_CLIENT_SECRET:
 - + Mở file credentials.json.
 - + Copy giá trị của client_id và client_secret.
- Lấy SUPABASE_URL và SUPABASE_SERVICE_KEY:
 - + Vào Supabase dashboard.
 - + Settings → API.
 - + Copy "Project URL" và "service_role" key.
- Tạo JWT_SECRET:

```
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

Copy output và paste vào .env.

Bước 4: Bảo mật file .env

- Quan trọng: File .env chứa thông tin nhạy cảm, KHÔNG được commit lên Git.
- Kiểm tra file .gitignore có dòng sau:

```
.env
credentials.json
```

Nếu chưa có thì thêm vào.

*2.2.8. Khởi chạy Backend Server**Bước 1: Đảm bảo Virtual Environment đang active*

- Kiểm tra: dòng (venv) phải xuất hiện. Nếu chưa active:

```
# Windows: .\venv\Scripts\activate
# macOS/Linux: source venv/bin/activate
```

Bước 2: Chạy Backend Server

```
# Di chuyển vào thư mục backend (nếu chưa)
# Khởi động server với auto-reload (development mode)
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

- + app.main:app - Import object app từ file app/main.py.
- + --reload - Tự động restart khi code thay đổi.
- + --host 0.0.0.0 - Cho phép truy cập từ mạng LAN (không chỉ localhost).
- + --port 8000 - Chạy trên port 8000.

Bước 3: Xác nhận server đang chạy: Nếu thấy output tương tự:

```
INFO: Will watch for changes in these directories:
['D:\\TTTN_GIT\\AppLLM_AutoReplyEmail']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [12345] using StatReload
INFO: Started server process [67890]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Backend đã sẵn sàng.

2.3. Cài đặt frontend & khởi chạy

2.3.1. Yêu cầu hệ thống

Ngoài Visual Studio Code & Git, người dùng cần cài thêm Node.js và npm.

- Node.js phiên bản 18.0 trở lên (khuyến nghị 18.x LTS hoặc 20.x LTS)
- npm đi kèm với Node.js.
- Kiểm tra phiên bản đã cài đặt:

```
node --version  
npm --version
```

- Tải về và cài đặt:
 - + Truy cập: <https://nodejs.org/>.
 - + Tải phiên bản LTS (Long Term Support).
 - + Chạy file cài đặt và làm theo hướng dẫn.

2.3.2. Các bước cài đặt

Bước 1: Clone Repository từ GitHub

Mở Terminal/Command Prompt và chạy lệnh sau:

```
git clone https://github.com/Harry1720/AppLLM\_AutoReplyEmail\_FE.git  
# Di chuyển vào thư mục dự án  
cd AppLLM_AutoReplyEmail_FE
```

Bước 2: Di chuyển vào thư mục Frontend

```
cd frontend
```

Bước 3: Cài đặt Dependencies: Chạy lệnh sau để cài đặt tất cả các package cần thiết:

```
npm install
```

- Quá trình này sẽ:
 - + Tải về và cài đặt tất cả các dependencies từ file package.json.
 - + Tạo thư mục node_modules chứa các package.
 - + Tạo file package-lock.json để lock phiên bản dependencies.
- Các package chính được cài đặt:
 - + Next.js 16.0.3: Framework React cho production.

- + React 19.2.0: Thư viện UI.
- + React DOM 19.2.0: Package để render React vào DOM.
- + DOMPurify: Thư viện sanitize HTML để bảo mật.
- + TypeScript 5: Ngôn ngữ lập trình.
- + Tailwind CSS 4: Framework CSS.
- + ESLint: Công cụ lint code.
- Thời gian cài đặt: Khoảng 2-5 phút tùy vào tốc độ mạng.

2.3.3. Cấu hình biến môi trường

Bước 1: Tạo file cấu hình môi trường

```
# Copy file template  
cp .env.example .env.local
```

Trên Windows (nếu lệnh cp không hoạt động):

```
Copy-Item .env.example .env.local
```

Hoặc thủ công:

- Tạo file mới tên .env.local trong thư mục frontend
- Copy nội dung từ file .env.example sang file .env.local

Bước 2: Cấu hình các biến môi trường: Mở file .env.local bằng trình soạn thảo và cập nhật các giá trị sau:

```
# API Configuration  
# URL của backend API (mặc định là localhost port 8000)  
NEXT_PUBLIC_API_URL=http://localhost:8000  
  
# Google OAuth Configuration  
# Thay your-google-client-id bằng Client ID bạn đã tạo ở Google Cloud  
Console  
NEXT_PUBLIC_GOOGLE_CLIENT_ID=your-google-client-  
id.apps.googleusercontent.com  
  
# Redirect URI sau khi đăng nhập Google (không cần thay đổi nếu chạy local)  
NEXT_PUBLIC_GOOGLE_REDIRECT_URI=http://localhost:3000/auth/ca  
llback
```

2.3.4. Khởi chạy ứng dụng

- Chế độ Development (Phát triển).

```
npm run dev
```

- Ứng dụng sẽ:
 - + Khởi động development server.
 - + Chạy tại địa chỉ: <http://localhost:3000>.
 - + Tự động reload khi bạn thay đổi code (Hot Module Replacement).
 - + Hiển thị lỗi chi tiết trên trình duyệt nếu có.
- Mở trình duyệt và truy cập: <http://localhost:3000>.

2.3.5. Kiểm tra cài đặt

- Kiểm tra Frontend đang chạy: Sau khi chạy `npm run dev`, bạn sẽ thấy output tương tự:

```
▲ Next.js 16.0.3
- Local:      http://localhost:3000
- Environments: .env.local
✓ Ready in 2.5s
```

- Kiểm tra trang Landing Page
 - + Mở trình duyệt.
 - + Truy cập <http://localhost:3000>.
 - + Bạn sẽ thấy trang chủ với:
 - Logo Gmail và AI.
 - Tiêu đề "Quản lý Email Thông Minh với AI".
 - Nút "Bắt đầu với Google".
- Kiểm tra kết nối Backend (nếu có)
 - + Nếu bạn đã cài đặt và chạy backend:
 - Đảm bảo backend đang chạy.
 - Click nút "Bắt đầu với Google" trên trang chủ.
 - Nếu thành công, bạn sẽ được chuyển đến trang đăng nhập Google.

- + Nếu backend chưa chạy:
 - Frontend vẫn có thể mở và hiển thị giao diện.
 - Tuy nhiên, các tính năng liên quan đến API sẽ không hoạt động.
 - Bạn sẽ thấy lỗi kết nối trong Console của trình duyệt.

2.3.6. Lưu ý quan trọng

- File .env.local không được commit lên Git.
 - + File này chứa thông tin nhạy cảm.
 - + Đã được thêm vào .gitignore.
- Cần có Backend để ứng dụng hoạt động đầy đủ
 - + Frontend này cần kết nối với backend API.
 - + Backend xử lý authentication, Gmail API, AI suggestions.
 - + Xem file SETUP.md để biết cách cài đặt backend.
- Development vs Production
 - + npm run dev: Dùng cho development, có hot reload.
 - + npm run build → npm run start: Dùng cho production, đã optimize
- Browser Support.
 - + Khuyến nghị: Chrome, Firefox, Safari, Edge phiên bản mới nhất.
 - + Cần bật JavaScript.

2.3.7. Quy trình khởi chạy nhanh

Sau khi đã cài đặt xong lần đầu, các lần sau bạn chỉ cần:

- Di chuyển vào thư mục frontend

```
cd AppLLM_AutoReplyEmail_FE/frontend
```

- Chạy development server

```
npm run dev
```

- Mở trình duyệt tại <http://localhost:3000>